

Task 05: Generative Adversarial Networks for MNIST

Akram Aki

June 1, 2026

1 Introduction

The goal of this task was to train Generative Adversarial Networks (GANs) that generate MNIST-like handwritten digit images. The main comparison is between a fully connected MLP GAN and a convolutional GAN. This comparison is useful because the MLP model is a simple baseline that ignores the spatial structure of the image, while the convolutional model can learn local stroke patterns directly. Both models were trained on normalized MNIST images, and generated samples were saved during training to study how the image quality changed over time.

2 Method

MNIST images were transformed to tensors and normalized to approximately the range $[-1, 1]$. This matches the final `tanh` activation of the generators. Both GANs used the same basic training setup:

$$\dim(z) = 128, \quad \text{batch size} = 128, \quad \text{epochs} = 40.$$

They were trained with binary cross entropy with logits for adversarial training. The MLP GAN uses fully connected layers and treats each image as a flattened vector of length $28 \cdot 28 = 784$. Its generator maps the latent vector through fully connected layers to a flattened image, while its discriminator maps the flattened image to a single real/fake logit:

$$\begin{aligned} G_{\text{MLP}} &: 128 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 784, \\ D_{\text{MLP}} &: 784 \rightarrow 256 \rightarrow 128 \rightarrow 1. \end{aligned}$$

The convolutional GAN keeps the spatial image structure and uses convolutional layers in the discriminator and image-shaped outputs in the generator. Here, the generator first projects the latent vector to a feature map and then upsamples it to an image, while the

discriminator uses convolutional blocks before the final output logit:

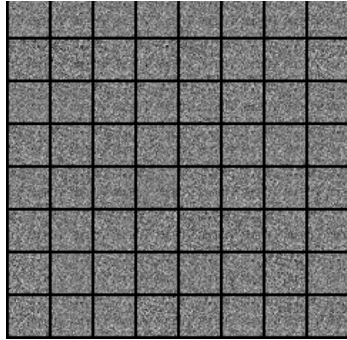
$$\begin{aligned} G_{\text{Conv}} &: 128 \rightarrow 128 \times 7 \times 7 \rightarrow 1 \times 28 \times 28, \\ D_{\text{Conv}} &: 1 \times 28 \times 28 \rightarrow 64 \rightarrow 128 \rightarrow 1. \end{aligned}$$

To improve training stability, the discriminator learning rate was chosen smaller than the generator learning rate, Adam with GAN-friendly momentum parameters was used, and real labels were smoothed. In practice, training without this balancing led to problems because the discriminator became too strong too quickly. Therefore, real MNIST images were labeled as 0.9 instead of 1.0 when training the discriminator, which prevents it from becoming overconfident too early. The same fixed noise vectors were used to generate progress images at different epochs.

3 Results

Figure 1 shows generated samples for the MLP and convolutional GANs before training, after an intermediate training stage, and at the end of training. At epoch 0, both models produce random noise. During training, both models begin to generate more structured digit-like images. The MLP GAN improves, but the generated digits remain noisy and are not always clearly recognizable. The convolutional GAN performs better and produces more coherent digit shapes, showing that preserving image structure is useful for this task.

Figure 2 compares the loss curves of the two standard GANs. The losses are useful for detecting obvious instabilities, but the generated samples are the most important result for this task. The convolutional GAN gives the best visual result in the main comparison.



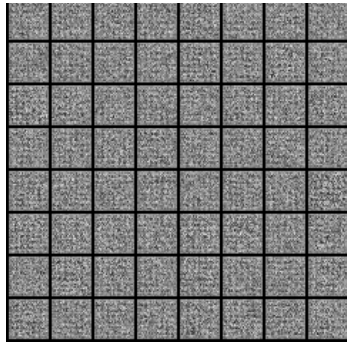
(a) MLP, epoch 0.



(b) MLP, epoch 20.



(c) MLP, epoch 40.



(d) Conv, epoch 0.



(e) Conv, epoch 20.



(f) Conv, epoch 40.

Figure 1: Generated samples from the MLP GAN and convolutional GAN during training. The convolutional model produces clearer and more recognizable digit-like samples.

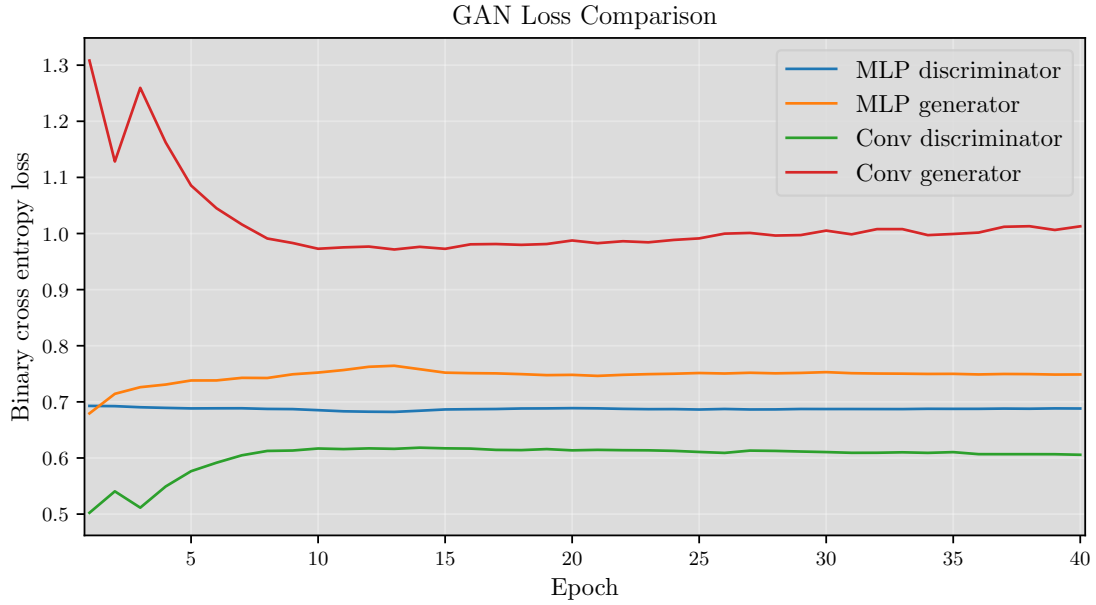


Figure 2: Training losses for the MLP and convolutional GANs. The loss curves should be interpreted together with the generated samples, since GAN losses are not expected to decrease monotonically.

4 Bonus: WGAN-GP

As an additional experiment, a Wasserstein GAN with Gradient Penalty (WGAN-GP) was trained. In a WGAN, the discriminator is replaced by a critic that assigns real-valued scores instead of probabilities. The gradient penalty was used instead of weight clipping to stabilize the critic. The WGAN-GP model also used a deeper convolutional architecture with 3×3 convolution kernels. The generator starts from the same latent dimension and uses progressively smaller feature maps before producing the final image, while the critic uses progressively wider convolutional layers:

$$\begin{aligned}
 G_{\text{WGAN-GP}} &: 128 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 1, \\
 C_{\text{WGAN-GP}} &: 1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 1, \\
 \text{kernel size} &= 3 \times 3, \quad \text{epochs} = 60.
 \end{aligned}$$

The WGAN-GP losses should not be directly compared numerically to the binary cross entropy losses of the MLP and convolutional GANs, because the objectives have different meanings. The comparison in Figure 3 is therefore mainly used to show training behavior, not to rank the models by loss value.

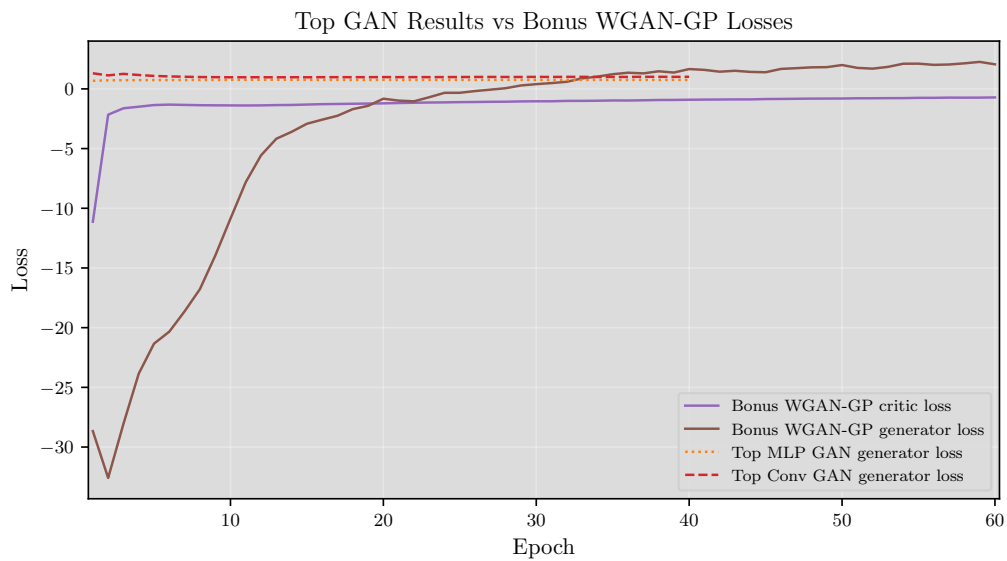
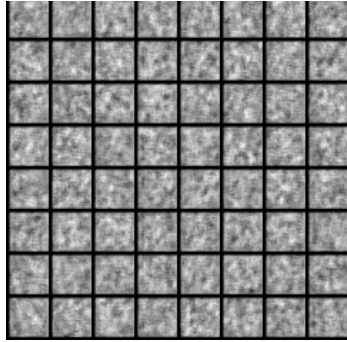


Figure 3: Loss curves for the bonus WGAN-GP together with the generator losses from the main GAN models. The WGAN-GP objective is different from the BCE GAN objective, so the absolute values are not directly comparable.

Figure 4 shows the WGAN-GP samples during training. The images start as noise, then become increasingly structured. Compared with the earlier models, the final WGAN-GP samples are visually closer to MNIST digits, with more coherent strokes and less random speckle.



(a) Epoch 0.



(b) Epoch 1.



(c) Epoch 15.



(d) Epoch 30.



(e) Epoch 45.



(f) Epoch 60.

Figure 4: Progression of generated samples from the WGAN-GP bonus model.

Figure 5 compares final samples from the three generators. For this comparison, the same set of latent noise vectors was passed to the MLP GAN, convolutional GAN, and WGAN-GP generator. The MLP model remains quite noisy and does not consistently generate clear digits. The convolutional GAN produces better and more recognizable digit shapes. The WGAN-GP samples look most similar to MNIST data, with better stroke thickness and a more diverse set of digit-like shapes, although diversity would need to be tested more systematically.

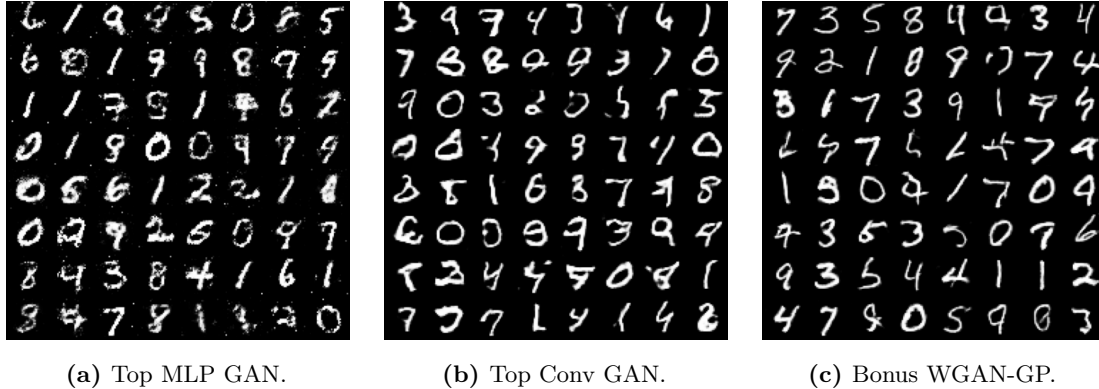


Figure 5: Final visual comparison using the same latent noise vectors for all three generators.

5 Conclusion

All trained models learned to generate MNIST-like samples to some degree. The MLP GAN demonstrates the basic adversarial setup, but its samples remain noisy. The convolutional GAN clearly improves over the MLP version and produces more recognizable digits. The WGAN-GP bonus model gives the best visual results in this study, although it was treated as an additional experiment rather than the main task.

Possible improvements include longer training, more systematic hyperparameter tuning, and quantitative metrics for sample quality and diversity. Examples could include classifier-based evaluation, nearest-neighbor checks against the training set, or more formal generative quality metrics.